GLADIUS Primer

Kang, Yoganarasimhan, and Jain (2025)

EconIRL Technical Notes

1 Problem and Motivation

Structural DDC estimation and inverse reinforcement learning share the same objective: recover a reward function from observed sequential decisions. NFXP (Rust, 1987) solves the Bellman equation exactly but scales as $O(|\mathcal{S}|^2)$ per iteration. CCP (Hotz and Miller, 1993) eliminates the inner loop but requires estimating transitions. AIRL (Fu et al., 2018) avoids transition estimation but has no convergence guarantees.

Kang, Yoganarasimhan, and Jain (2025) propose GLADIUS (Gradient-based Learning with Ascent-Descent for Inverse Utility learning from Samples), a single-loop gradient method that jointly minimizes negative log-likelihood and Bellman error without estimating transitions. The Bellman residual satisfies the Polyak-Łojasiewicz (PL) condition, ensuring global convergence. GLADIUS is the first offline IRL/DDC estimator that is simultaneously transition-free, gradient-based, globally convergent, and scalable. It achieves $1/\sqrt{N}$ statistical complexity matching NFXP and CCP.

A dual variable $\zeta(s,a) \approx \mathbb{E}[V(s')|s,a]$ resolves the double-sampling problem (where naive TD error includes irreducible transition variance). The Q-network descends on NLL while ζ ascends to tighten the Bellman bound. Rewards are recovered as $r(s,a) = Q(s,a) - \beta\,\zeta(s,a)$.

2 Model and Notation

An agent in state $s \in \mathcal{S}$ chooses $a \in \mathcal{A}$ to maximize entropy-regularized discounted utility. Under T1EV errors this is equivalent to the MaxEnt-IRL objective:

$$\max_{\pi}\ \mathbb{E}\big[\textstyle\sum_{t=0}^{\infty} \beta^t \big(r(s_t,a_t) + \mathcal{H}(\pi(\cdot \mid s_t))\big)\big]\,. \tag{1}$$

The optimal policy is $\pi^*(a|s) = \exp(Q^*(s,a))/\sum_{a'}\exp(Q^*(s,a'))$ with scale $\sigma = 1$.

Table 1: MDP objects and GLADIUS network components.

Symbol	Name	Definition / Role
$r(s,a)$	Reward	Per-period utility (structural primitive to recover)
$Q(s,a)$	Choice-specific value	$r(s,a) + \beta\,\mathbb{E}[V(s') \mid s,a]$
$V(s)$	Integrated value	$\log\sum_a \exp(Q(s,a))$ (logsumexp, $\sigma = 1$)
$\pi(a\|s)$	Policy (CCP)	$\exp(Q(s,a))/\sum_{a'}\exp(Q(s,a'))$
$P(s'\|s,a)$	Transitions	Unknown; never estimated
a_s	Anchor action	Action with known reward $r(s,a_s)$ at each state
$Q_{\theta_2}(s,a)$	Q-network	MLP mapping (state features, action) $\to$ scalar
$\zeta_{\theta_1}(s,a)$	EV-network	MLP approximating $\mathbb{E}[V(s') \mid s,a]$; **not** $V(s)$

There are only two networks, not three. Unlike AIRL, GLADIUS does not maintain a separate policy network. The policy is derived analytically from Q via softmax. This is possible because the Bellman error term implicitly handles the forward-looking component that AIRL delegates to PPO.

Loss components. The expected risk combines two terms:

$$\mathcal{L}_{\text{NLL}}(Q) = -\mathbb{E}\left[\log \frac{\exp(Q(s,a))}{\sum_{a'} \exp(Q(s,a'))}\right], \tag{2}$$

$$\mathcal{L}_{\text{BE}}(Q) = \mathbb{E}\big[(Q(s,a) - r(s,a) - \beta\, V_Q(s'))^2\big]. \tag{3}$$

The NLL term fits the policy to observed choices. The Bellman error enforces structural consistency between Q and the transition dynamics. Naive minimization of Eq. (3) suffers from the double-sampling problem: $\mathbb{E}[(V_Q(s') - \mathbb{E}[V_Q(s')|s,a])^2]$ is irreducible variance that biases the gradient. GLADIUS resolves this with a dual variable.

Unbiased Bellman error. Introducing $\zeta(s,a) \approx \mathbb{E}[V_Q(s')|s,a]$ and taking the maximum over ζ:

$$\mathcal{L}_{\text{BE}}(Q) = \max_{\zeta}\ -\mathbb{E}\big[(V_Q(s') - \zeta(s,a))^2\big]. \tag{4}$$

At the optimum, $\zeta^*(s,a) = \mathbb{E}[V_Q(s')|s,a]$, and the objective equals $-\text{Var}(V_Q(s')|s,a)$, canceling the bias.

GLADIUS objective. Combining Eqs. (2) and (4):

$$\min_{Q} \max_{\zeta}\ \mathcal{L}_{\text{NLL}}(Q)\ -\ \mathbb{E}\big[(V_Q(s') - \zeta(s,a))^2\big]. \tag{5}$$

This is the minimax problem solved by alternating gradient ascent (on ζ) and descent (on Q).

Reward recovery. After convergence: $\hat{r}(s,a) = \hat{Q}(s,a) - \beta\, \hat{\zeta}(s,a)$.

Additional notation. $\mathcal{D} = \{(s_i, a_i, s_i')\}_{i=1}^N$ is the offline dataset of observed transitions. θ_1 parameterizes ζ and θ_2 parameterizes Q. τ_1, τ_2 are learning rates. T is the number of gradient steps.

3 Identification

Anchor action (Magnac and Thesmar, 2002). For all s, there exists $a_s \in \mathcal{A}$ with known $r(s, a_s)$. In the Rust bus, this is the replacement action ($a = 1$) with known cost. The anchor pins the level of Q: since $Q(s,a_s) = r(s,a_s) + \beta\, \mathbb{E}[V(s')|s,a_s]$, knowing $r(s,a_s)$ determines $Q(s,a_s)$ given $\zeta(s,a_s)$, and the remaining $Q(s,a)$ are identified through the NLL term.

Theorem 1 (Identification; Kang et al. 2025, Thm. 3)**.** *Under the anchor action assumption, the minimizer of the expected risk $\mathcal{R}_{exp}(Q) = \mathcal{L}_{NLL}(Q) + \mathcal{L}_{BE}(Q)$ uniquely identifies Q^* on the support of the data. The reward is then recovered as $r(s,a) = Q^*(s,a) - \beta\, \mathbb{E}[V_{Q^*}(s')|s,a]$.*

Theorem 2 (Global convergence; Kang et al. 2025, Thm. 11)**.** *Both $\mathcal{L}_{NLL}$ and $\mathcal{L}_{BE}$ satisfy the Polyak-Łojasiewicz condition. After T steps of stochastic gradient descent-ascent: $\mathbb{E}[dist(\hat{\theta}_T, \Theta^*)] = O(T^{-1/4})$. The statistical complexity is $O(1/\sqrt{N})$, matching NFXP and CCP.*

Structural identification bias in IRL. When rewards are not observed, Q is trained by NLL only (behavioral cloning), identifying Q up to a state-dependent constant $c(s)$. This constant propagates asymmetrically through the transition structure into implied rewards. On the Rust bus with $\beta = 0.95$, GLADIUS recovers replacement cost within 8% but overestimates operating cost by about 40%. This is a structural bias, not a tuning problem. It disappears when rewards are observed (the paper's intended use case).

Comparison to AAIRL anchors. Both GLADIUS and AAIRL use anchor normalizations, but differently. AAIRL anchors the exit action at zero reward and the absorbing state at zero value, which pins the shaping potential h_ϕ and identifies action-dependent rewards. GLADIUS anchors a known-reward action at each state, which pins the Q-level and identifies rewards through the Bellman decomposition. AAIRL requires the discriminator to converge to the population optimum. GLADIUS has a provable convergence guarantee via the PL condition.

4 Estimation Algorithm

Algorithm 1: GLADIUS: Gradient-based Learning with Ascent-Descent

Input: Dataset $\mathcal{D} = \{(s_i, a_i, s_i')\}_{i=1}^N$, discount β, learning rates τ_1, τ_2, iterations T

Initialize Q_{θ_2}, ζ_{θ_1} (neural networks)

for $t = 1, \ldots, T$ **do**

  Draw mini-batches $B_1, B_2 \sim \mathcal{D}$

  `// Ascent step: update` ζ `(fix` Q`)`

  $V_Q(s') \leftarrow \log \sum_{a'} \exp(Q_{\theta_2}(s', a'))$ for each $s' \in B_1$

  $\mathcal{L}_\zeta \leftarrow -\frac{1}{|B_1|} \sum_{B_1} (V_Q(s') - \zeta_{\theta_1}(s, a))^2$

  $\theta_1 \leftarrow \theta_1 + \tau_1 \nabla_{\theta_1} \mathcal{L}_\zeta$ `// ascent on` ζ

  `// Descent step: update` Q `(fix` ζ`)`

  $\log p_Q(a|s) \leftarrow Q_{\theta_2}(s, a) - \log \sum_{a'} \exp(Q_{\theta_2}(s, a'))$ for each $(s, a) \in B_2$

  $\mathcal{L}_Q \leftarrow -\frac{1}{|B_2|} \sum_{B_2} \log p_Q(a|s)$ `// NLL only in IRL setting`

  $\theta_2 \leftarrow \theta_2 - \tau_2 \nabla_{\theta_2} \mathcal{L}_Q$ `// descent on` Q

end

`// Reward recovery`

$\hat{r}(s, a) \leftarrow Q_{\theta_2}(s, a) - \beta\, \zeta_{\theta_1}(s, a)$ for all (s, a)

$\hat{\pi}(a|s) \leftarrow \mathrm{softmax}(Q_{\theta_2}(s, \cdot))$

Output: $\hat{r}$, $\hat{Q}$, $\hat{\zeta}$, $\hat{\pi}$

Implementation notes. In the IRL setting (no observed rewards), Bellman gradients are **not** passed through Q in the descent step. Without observed rewards to anchor the scale, gradients through $V_Q = \mathrm{logsumexp}(Q)$ cause Q-value explosion. Instead, Q is trained by NLL only (softmax behavioral cloning), and Bellman consistency is enforced exclusively through the ζ ascent step. When rewards are observed, the full Bellman gradient can be used (Algorithm 2 in the paper). The EconIRL implementation uses alternating updates: even mini-batches update ζ, odd mini-batches update Q. Learning rates decay as $\tau_t = \tau_0/(1 + 0.001\, t)$. Early stopping with patience 50 halts training when loss does not improve by 10^{-6} for 50 consecutive epochs.

Action-difference projection. The implied reward $\hat{r}(s, a)$ contains an unidentified state-dependent constant $c(s)$ that cancels in choice probabilities but biases reward levels. To extract structural parameters θ, GLADIUS projects reward differences onto feature differences: for each action $a > 0$ relative to baseline action 0, regress $\hat{r}(s, a) - \hat{r}(s, 0)$ on $\phi(s, a) - \phi(s, 0)$ by least squares.

5 Results

Table 2: GLADIUS recovery on Rust bus (90 bins, $\beta = 0.95$, 200×100).

	True	GLADIUS	NFXP
Op. cost	1.000	—	—
Repl. cost	5.000	—	—
Cosine sim.	—	—	—
Policy match (%)	—	—	—
Time (sec)	—	—	—

On the Rust bus with $\beta = 0.95$ and 200 agents, GLADIUS recovers replacement cost within 8% and achieves near-perfect policy match. Operating cost is overestimated by roughly 40% due to the structural identification bias in the IRL setting (see Section 3). NFXP recovers both within 5%. The bias disappears when rewards are observed.

From Kang et al. (2025): with K irrelevant dummy variables expanding the state space, GLADIUS maintains MAPE below 10% even at $K = 100$ ($20^{100} \approx 10^{130}$ states), where NFXP and CCP are infeasible. GLADIUS complements NFXP (exact but non-scalable) and AIRL (scalable but no convergence guarantee).

6 EconIRL Implementation

The estimator is in `src/econirl/estimation/gladius.py` as `GLADIUSEstimator`. The neural variant with context encoders is in `src/econirl/estimators/neural_gladius.py`. Run `python papers/primers/gladius_example.py` to regenerate results.

Table 3: Code mapping.

Step	Method	Operation
ζ ascent	`zeta_step()`	$\mathrm{MSE}(V_Q(s') - \zeta(s, a))^2$
Q descent	`q_step()`	NLL softmax (no Bellman grad)
Reward	`_extract_parameters()`	$\hat{r} = Q - \beta\zeta$; action-diff projection
Policy	softmax(Q)	Derived analytically, no separate network

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.gladius import GLADIUSEstimator, GLADIUSConfig

env = RustBusEnvironment(operating_cost=1.0, replacement_cost=5.0,
                         num_mileage_bins=90, discount_factor=0.95)
panel = env.generate_panel(n_individuals=200, n_periods=100, seed=42)

gladius = GLADIUSEstimator(config=GLADIUSConfig(
    q_hidden_dim=64, v_hidden_dim=64, max_epochs=500,
    alternating_updates=True))
```

```
result = gladius.estimate(panel=panel, utility=LinearReward.
    from_environment(env),
                              problem=env.problem_spec, transitions=env.
                                  transition_matrices)
```

Table 4 compares GLADIUS to other estimators in the EconIRL package. GLADIUS is the only method that is simultaneously transition-free, gradient-based, globally convergent, and scalable. AAIRL shares the transition-free and scalable properties but lacks convergence guarantees and statistical complexity results. NFXP and CCP satisfy the Bellman equation exactly but require known transitions and do not scale to high-dimensional states.

Table 4: GLADIUS in context. Checkmarks indicate the property holds by design.

	NFXP	CCP	AIRL	AAIRL	GLADIUS
Transition-free			✓	✓	✓
Gradient-based					✓
Global convergence	✓	✓			✓
Scalable (neural)			✓	✓	✓
Bellman satisfied	✓	✓			✓
Action-dep. rewards	✓	✓		✓	✓
Heterogeneity				✓	
Stat. complexity	$1/\sqrt{N}$	$1/\sqrt{N}$	—	—	$1/\sqrt{N}$

Table 5 lists the default configuration. The most important setting is `alternating_updates`: setting it to `False` in the IRL setting (no observed rewards) causes Q-value explosion because gradients through $V_Q = \text{logsumexp}(Q)$ are unbounded without a reward anchor. The `value_scale` parameter is auto-computed as $1/(1-\beta)$ and prevents numerical overflow at high discount factors.

Table 5: Default configuration and tuning guidance.

Parameter	Default	Notes
`q_hidden_dim`	128	Width of Q-network hidden layers
`v_hidden_dim`	128	Width of ζ-network hidden layers
`q_num_layers` / `v_num_layers`	3	Depth of both networks
`q_lr` / `v_lr`	10^{-3}	Learning rates for Q and ζ
`batch_size`	512	Mini-batch size
`max_epochs`	500	Upper bound on training iterations
`patience`	50	Early stopping window
`alternating_updates`	True	Critical for IRL (prevents Q explosion)
`bellman_penalty_weight`	1.0	Weight on $\mathcal{L}_{\text{BE}}$ relative to $\mathcal{L}_{\text{NLL}}$
`gradient_clip`	1.0	Max gradient norm
`value_scale`	auto	Set to $1/(1-\beta)$; stabilizes high-β settings